Linux Fundamentals

Workshop notes — Cybersecurity Workshop

What is an operating system?

An Operating System (OS) is software that manages and handles the hardware and software resources of a computing device.

- **Manages the hardware:** translates software commands into actions your hardware understands — for example, turning a mouse click into the exact instructions your hardware needs to carry it out.
- **Resource allocation:** lets you run multiple applications at the same time.
- **Security:** isolates applications from each other — if one program crashes, it doesn't bring the whole system down with it.

Examples: Windows, Linux, macOS, Android...

Why Linux?

- **Control:** Windows hides complexity from you, which also means less control over your own machine. Linux gives you full control — you can even delete the entire system with a single command.
- **Everywhere:** Linux runs behind the scenes almost everywhere — around 90% of the world's cloud infrastructure, most servers, Android phones, and IoT devices like smart routers and cameras.
- **Open source:** the source code is public on GitHub. Anyone can read it, and anyone can contribute to it — unlike Windows.
- **Developer-friendly:** it's an easier environment to build and work in.
- **Built for security work:** most security tools (Nmap, Metasploit...) are made to run on Linux.

Linux distros

A Linux distro (distribution) is the complete software environment built on the Linux kernel, packaged with the system tools and software you need. Some distros also come with a desktop environment; others don't — they're pure command line.

Examples:

- **Fedora:** ships the latest versions of software, good for developers who want cutting-edge tools.
- **Ubuntu:** beginner-friendly and easy to install, great for general use.
- **Kali Linux:** built for cybersecurity, comes preloaded with 600+ security tools.

Tonight's challenges run on a distro with no desktop environment at all — everything happens through the shell.

Linux file tree

What is a file system?

A file system is the method an operating system uses to store, organize, and manage files on a storage device.

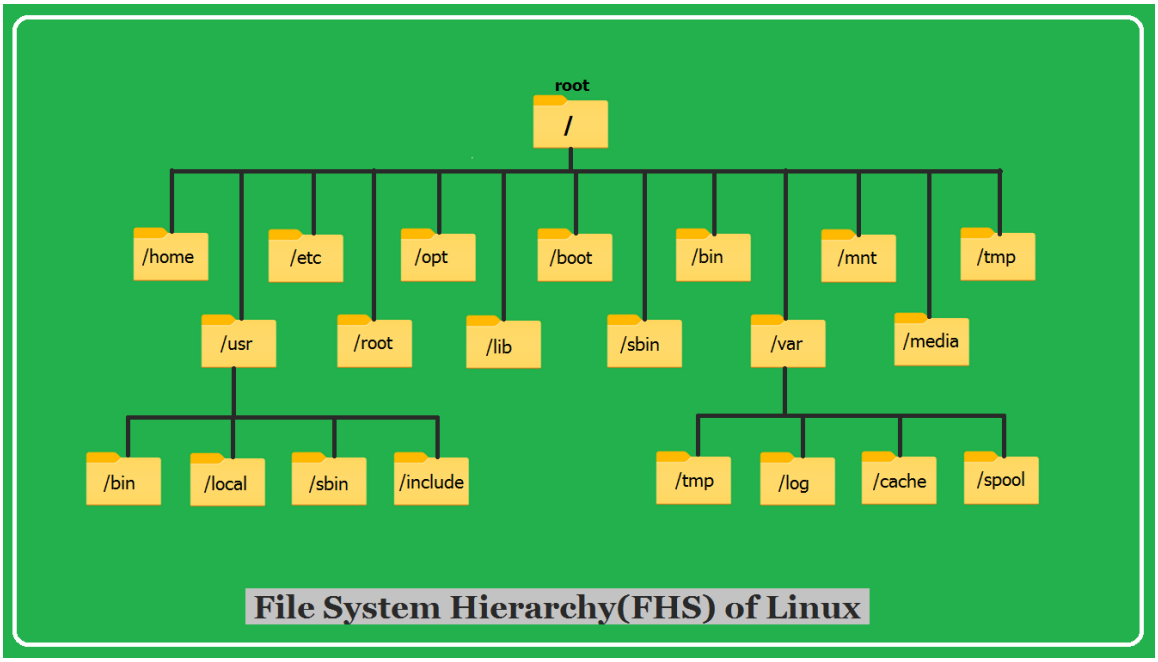
Comparison with Windows

On Windows, storage is split across multiple drives, like **C:\ , D:\ ...**



Windows drives shown in File Explorer

On Linux, all storage lives under a single directory called **/** (the root directory). A folder on Linux is called a **directory**.



Linux root directory tree

These are the important directories to know for now:

- **/root:** the home directory of the root user (more on root later).
- **/home:** contains each user's personal directory — their documents, files, and settings.
- **/bin:** essential commands any user can run, like ls, cd, mkdir, find.
- **/etc:** system configuration files.
- **/opt:** where extra, optional software gets installed — you'll run into this a lot tonight.

- **/tmp:** temporary files, usually cleared automatically.
- **/usr:** installed programs and shared resources (e.g. /usr/local/bin, another common home for installed programs).

For additional details on linux directories

Shell

A **shell** is the program that lets you control your computer by typing commands instead of clicking around. **Terminal** and **command line interface (CLI)** are basically the same thing, and the three terms get used interchangeably — the small technical differences between them don't matter for today.

In simple terms, the terminal is the black window where you type commands, and the shell is what reads what you typed and actually runs it.



shell picture

Shell commands

A shell command is a text instruction you type to make the computer do something. Most commands follow the same pattern:

command → **target** → **flags**

Take `mkdir test_dir`:

- `mkdir` is the **command** — create a directory.
- `test_dir` is the **target** — the name to give it.

> **Note:** not every command needs a target or a flag. Some work fine on their own, or fall back to a default target if you don't give one.

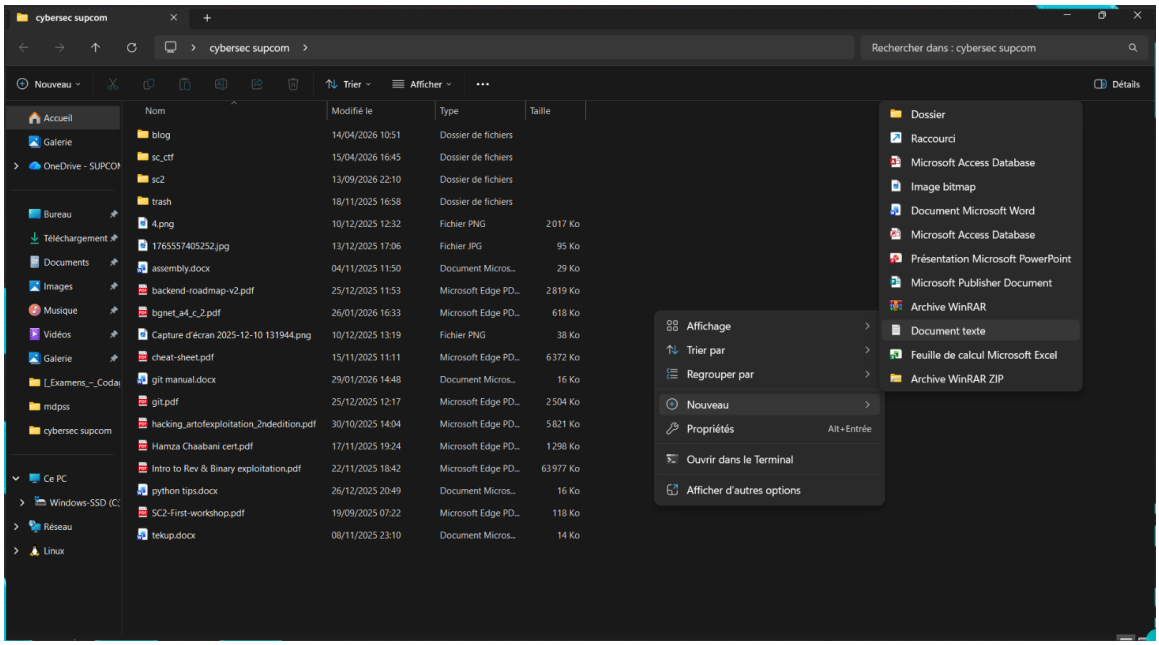
Let's chain a few commands together and see what happens, step by step:

```
mkdir test_dir      # create a directory called test_dir
cd test_dir         # move into it
touch file.txt      # create a file called file.txt inside it
pwd                 # print your current location
ls                  # show what's inside the directory
ls -l               # show more details about each item (more on this later)
```

Practice

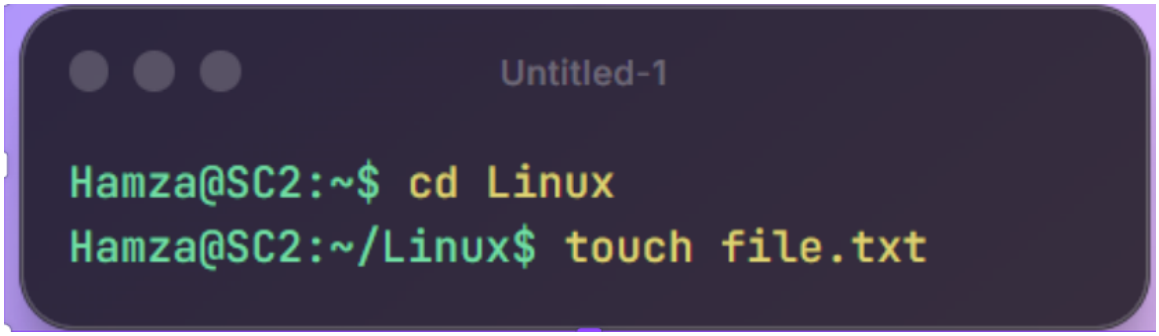
The same action — creating a file — looks very different depending on the OS.

On Windows, you'd right-click inside a folder → New → Document:



Creating a new file in Windows via right-click

On Linux, you just ran `touch file.txt` above — no mouse required:



Creating a file in Linux via the terminal

Basic linux commands

Users & groups

In Linux, **users** and **groups** are the core mechanism used to control system access, organize file ownership, and manage permissions.

Anyone using a computer needs their own account — for their own documents, programs, and settings. That way, Bill and Mary can share the same machine without touching each other's files. Each person gets their own account, under whatever username they like.

Now picture a company: Alice works in HR, Bob in engineering, and Charlie in finance. All their files live on one shared server, but not everyone should be able to touch everything:

Folder	Full access (read & write)	Read-only
/company_data/employee_records/	Admin, HR	—
/company_data/source_code/	Admin, Engineering	—

/company_data/payroll/	Admin, Finance	Everyone else

Instead of setting permissions file by file for every single person, we add each person to the right group and set permissions once for the whole group.

Users

`root` is the default super-user on every Linux system — it has full access to everything, including the ability to change or delete system files.

Note:
• /etc/passwd stores basic account info (username, user ID, home directory, shell) — passwords are no longer stored here.
• /etc/shadow stores the actual encrypted passwords. Only root can read it.
• whoami prints the user you're currently logged in as.

Groups

A group is a set of users who share the same permissions — for example, only the `developers` group might be allowed to edit a certain file.

Every user automatically gets their own group, named after them. A user can belong to several groups, but always has one primary group. Being added to a group can give you access to things you wouldn't normally have — that becomes important later.

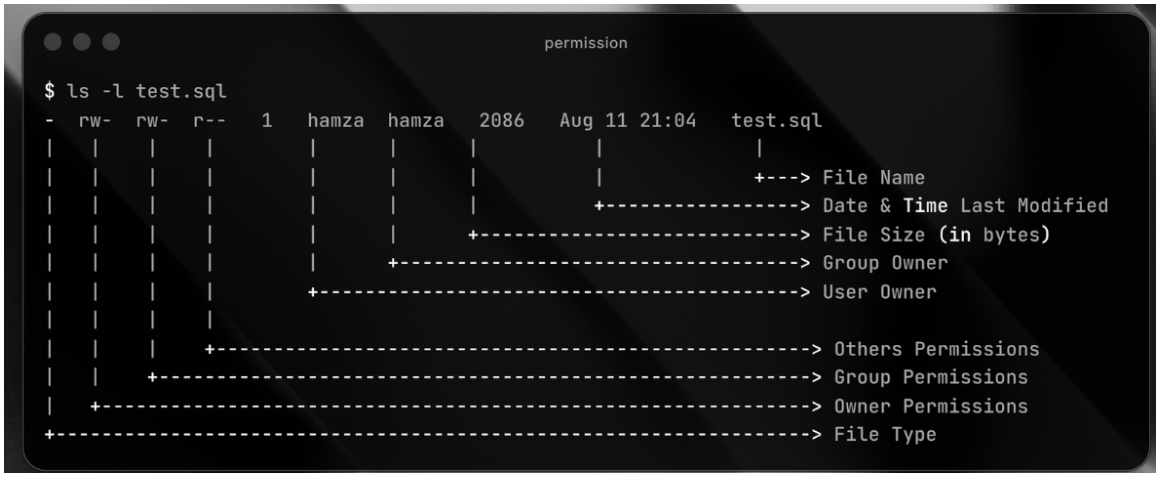
[For additional resources on users and groups check this out.]

Permissions

Each file has three permissions: read (r), write (w), execute (x).

On a file, execute (x) means you can run it. On a directory, execute (x) means you can enter it — without it, you can't cd into that folder even if you can see it.

(add additional resources)



Output of ls -l test.sql

In the picture above, this is the output of `ls -l test.sql`. The owner, **hamza**, and his primary group both have *read* and *write* permissions, while **others** only have *read*.

Note: the owner of a file can change its permissions however they like.

For more information on permissions, check this

Sudo

Normal users sometimes need to run a command that requires root privileges — for example, installing a new program or restarting a system service. Giving everyone the root password would mean everyone has full, unrestricted access to the machine, with no way to know who did what. Instead, we use sudo to let specific users run specific commands with root privileges, using their own password instead of root's.

Sudo rules are defined in `/etc/sudoers.d/` (a folder of rule files). Example:

```
alice ALL=(root):/usr/bin/python3
```

This limits Alice to running Python with root privileges — nothing else.

General syntax:

```
user_or_group  host=(runas_user:runas_group)  commands
```

Standard admin rule:

```
%admin   ALL=(ALL:ALL) ALL
```

- `%admin`: the % means this rule applies to a whole group (here, admin), not just one user.
- `ALL=` (1st): applies on **all hosts**.
- `(ALL:ALL)`: can run commands as **any user**, in **any group**.
- `ALL` (2nd): can run **any command**.

Quick distinction: `su` switches your whole session to another user and asks for *their* password. `sudo` runs a single command as another user (usually root) and asks for *your own* password.

For more information about sudo